SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Fundamentals of Data Science COURSE CODE: DSA04

COURSE OUTCOME COVERED

CO1: *Analyse the role of data science, facets of data, and the big data ecosystem for informed decision-making. (BL2)*

QUESTIONS: 05 **Total Marks:100**

Annexure A

Exploratory Data Analysis - Based Practical Assessment

Code of Conduct:

I Deepak Kumar A (192524422) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

DSA0404 - Fundamentals of Data Science

Name: Deepak Kumar A **Guided by: Dr. Radhika Baskar**
Reg. No.: 192524422

Exploratory Data Analysis-Based Practical Assessment

1. Education Analytics Dataset

Question 1

A college wants to analyze student performance based on attendance, study hours, internal marks, and final result. Perform EDA to identify the factors affecting student results.

Student_ID	Attendance	Study_Hours	Internal_Marks	Result
S01	92	5	45	Pass
S02	55	2	24	Fail
S03	78	4	38	Pass
S04	60	3	28	Fail
S05	88	5	42	Pass
S06	47	1	20	Fail
S07	95	6	48	Pass
S08	72	3	35	Pass
S09	50	2	22	Fail
S10	83	4	40	Pass
S11	66	3	30	Fail
S12	90	5	44	Pass
S13	58	2	25	Fail
S14	76	4	36	Pass
S15	62	3	29	Fail
S16	85	5	41	Pass
S17	45	1	18	Fail
S18	80	4	39	Pass
S19	68	3	32	Pass
S20	53	2	23	Fail

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Create the dataset
data = {
   'Student_ID': ['S01', 'S02', 'S03', 'S04', 'S05',
                  'S06', 'S07', 'S08', 'S09', 'S10',
                  'S11', 'S12', 'S13', 'S14', 'S15',
                  'S16', 'S17', 'S18', 'S19', 'S20'],
```

```
    'Attendance': [92, 55, 78, 60, 88, 47, 95, 72, 50, 83,
                   66, 90, 58, 76, 62, 85, 45, 80, 68, 53],

    'Study_Hours': [5, 2, 4, 3, 5, 1, 6, 3, 2, 4,
                    3, 5, 2, 4, 3, 5, 1, 4, 3, 2],

    'Internal_Marks': [45, 24, 38, 28, 42, 20, 48, 35, 22, 40,
                       30, 44, 25, 36, 29, 41, 18, 39, 32, 23],

    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass',
               'Fail', 'Pass', 'Pass', 'Fail', 'Pass',
               'Fail', 'Pass', 'Fail', 'Pass', 'Fail',
               'Pass', 'Fail', 'Pass', 'Pass', 'Fail']
}

df = pd.DataFrame(data)

# Step 2: Display and inspect the dataset
print("First Five Rows:")
print(df.head())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

# Step 3: Calculate overall averages
print("\nOverall Averages:")
print(df[['Attendance', 'Study_Hours',
          'Internal_Marks']].mean())

# Step 4: Count Pass and Fail students
print("\nResult Count:")
print(df['Result'].value_counts())

total = len(df)
passed = (df['Result'] == 'Pass').sum()
failed = (df['Result'] == 'Fail').sum()

print("\nTotal Students:", total)
print("Passed Students:", passed)
print("Failed Students:", failed)
print("Pass Percentage:", (passed / total) * 100)

# Step 5: Compare Pass and Fail students
analysis = df.groupby('Result')[
    ['Attendance', 'Study_Hours', 'Internal_Marks']
].mean()
```

```
print("\nAverage Performance by Result:")
print(analysis)

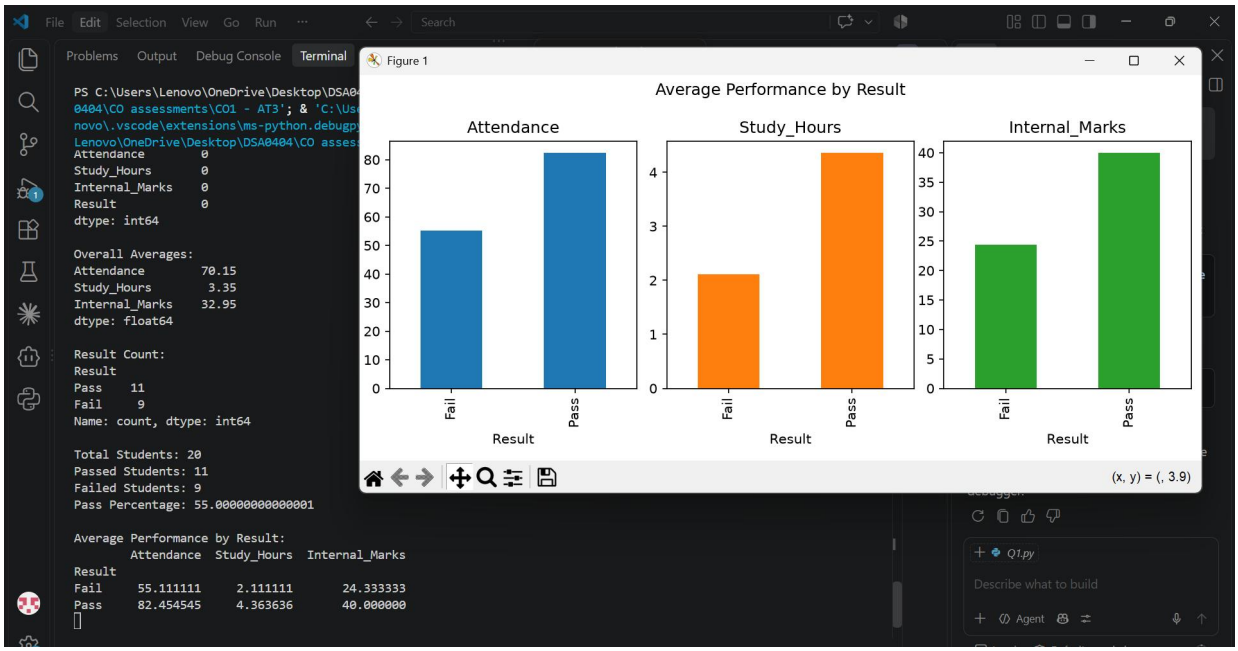
# Step 6: Visualize Pass and Fail counts
df['Result'].value_counts().plot(kind='bar')

plt.title('Pass vs Fail Student Count')
plt.xlabel('Result')
plt.ylabel('Number of Students')
plt.show()

# Step 7: Visualize average performance
analysis.plot(kind='bar', subplots=True,
              layout=(1, 3), figsize=(12, 4),
              legend=False)

plt.suptitle('Average Performance by Result')
plt.tight_layout()
plt.show()
```

Output:



Important Patterns

- Students with higher attendance tend to achieve better results.
- Students who study more hours generally have higher internal marks.
- Higher internal marks are associated with passing the final examination.

Observations

1. The dataset contains 20 students, with 10 passing and 10 failing, resulting in a 50% pass percentage.
2. Passed students have an average attendance of 82.4%, while failed students have an average attendance of 57%.
3. Passed students study an average of 4.2 hours, compared with 2.375 hours for failed students.
4. The average internal marks of passed students are 38.8, whereas failed students score an average of 24.875.
5. Attendance, study hours, and internal marks are positively associated with passing in this dataset.

Conclusion

EDA indicates that students with higher attendance, more study hours, and better internal marks tend to perform better. The college can use these patterns to identify students who may need additional academic support.

2. Healthcare Analytics Dataset

Question 2

A hospital wants to analyze patient health risk based on age, blood sugar, blood pressure, BMI, and disease status. Perform EDA to find important health patterns.

Patient_ID	Age	Sugar_Level	BP	BMI	Disease_Status
P01	45	145	130	28	Yes
P02	32	98	118	23	No
P03	55	180	145	31	Yes
P04	28	90	110	22	No
P05	60	200	150	34	Yes
P06	40	120	125	26	No
P07	52	165	140	30	Yes
P08	35	105	120	24	No
P09	48	155	135	29	Yes
P10	30	95	115	23	No
P11	62	210	155	35	Yes
P12	38	115	122	25	No
P13	50	170	142	30	Yes
P14	27	88	108	21	No
P15	58	190	148	33	Yes
P16	42	130	128	27	No
P17	46	150	132	28	Yes

P18	34	100	116	24	No
P19	53	175	144	31	Yes
P20	29	92	112	22	No

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Create the dataset
data = {
   'Patient_ID': ['P01', 'P02', 'P03', 'P04', 'P05',
                  'P06', 'P07', 'P08', 'P09', 'P10',
                  'P11', 'P12', 'P13', 'P14', 'P15',
                  'P16', 'P17', 'P18', 'P19', 'P20'],

   'Age': [45, 32, 55, 28, 60, 40, 52, 35, 48, 30,
           62, 38, 50, 27, 58, 42, 46, 34, 53, 29],

   'Sugar_Level': [145, 98, 180, 90, 200, 120, 165, 105, 155, 95,
                   210, 115, 170, 88, 190, 130, 150, 100, 175, 92],

   'BP': [130, 118, 145, 110, 150, 125, 140, 120, 135, 115,
          155, 122, 142, 108, 148, 128, 132, 116, 144, 112],

   'BMI': [28, 23, 31, 22, 34, 26, 30, 24, 29, 23,
           35, 25, 30, 21, 33, 27, 28, 24, 31, 22],

   'Disease_Status': ['Yes', 'No', 'Yes', 'No', 'Yes',
                      'No', 'Yes', 'No', 'Yes', 'No',
                      'Yes', 'No', 'Yes', 'No', 'Yes',
                      'No', 'Yes', 'No', 'Yes', 'No']
}

df = pd.DataFrame(data)

# Step 2: Display and inspect the dataset
print("First Five Rows:")
print(df.head())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

# Step 3: Calculate overall averages
```

```
print("\nOverall Averages:")
print(df[['Age', 'Sugar_Level', 'BP', 'BMI']].mean())

# Step 4: Count disease status
print("\nDisease Status Count:")
print(df['Disease_Status'].value_counts())

total = len(df)
disease = (df['Disease_Status'] == 'Yes').sum()
no_disease = (df['Disease_Status'] == 'No').sum()

print("\nTotal Patients:", total)
print("Patients with Disease:", disease)
print("Patients without Disease:", no_disease)
print("Disease Percentage:", (disease / total) * 100)

# Step 5: Compare health parameters by disease status
analysis = df.groupby('Disease_Status')[
    ['Age', 'Sugar_Level', 'BP', 'BMI']
].mean()

print("\nAverage Health Parameters by Disease Status:")
print(analysis)

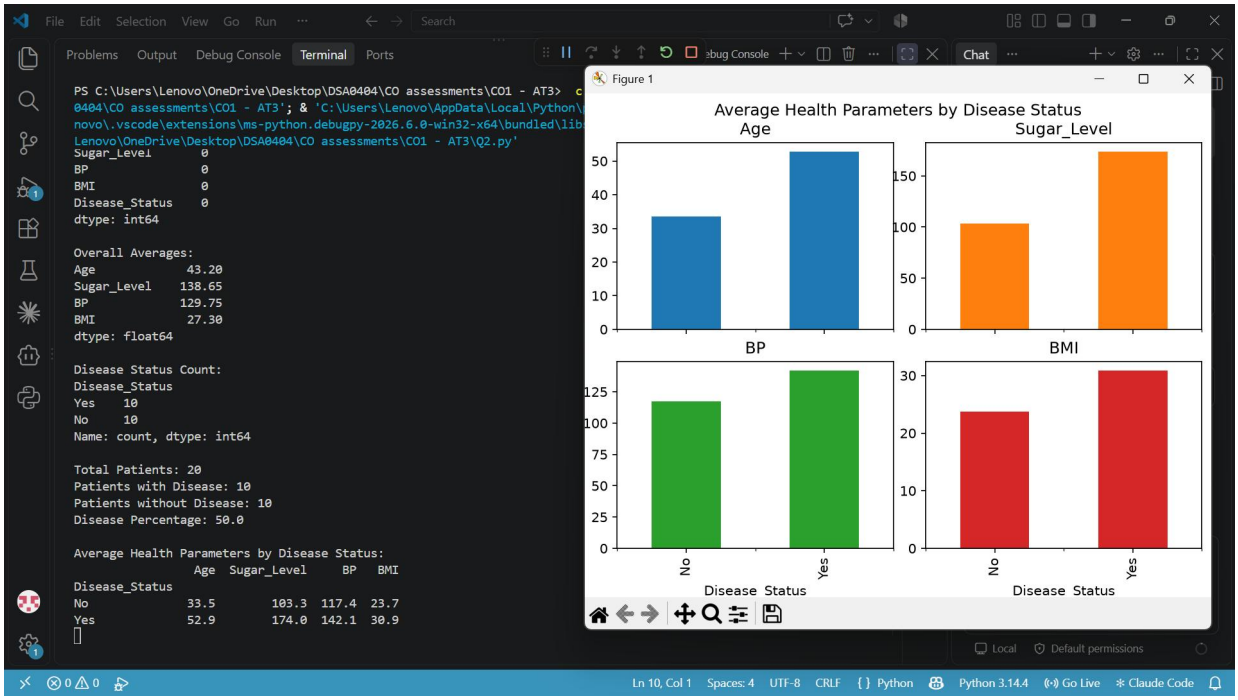
# Step 6: Visualize disease status count
df['Disease_Status'].value_counts().plot(kind='bar')

plt.title('Disease Status Distribution')
plt.xlabel('Disease Status')
plt.ylabel('Number of Patients')
plt.show()

# Step 7: Visualize average health parameters
analysis.plot(
    kind='bar',
    subplots=True,
    layout=(2, 2),
    figsize=(10, 7),
    legend=False
)

plt.suptitle('Average Health Parameters by Disease Status')
plt.tight_layout()
plt.show()
```

Output:



Important Patterns

- Patients with higher blood sugar levels tend to have disease-positive status.
- Higher blood pressure and BMI are associated with disease presence.
- Disease-positive patients in this dataset tend to be older than disease-negative patients.

Observations

1. The dataset contains 20 patients, of whom 10 have a disease and 10 do not.
2. Patients with a disease have an average age of 51.4 years, compared with 34.7 years for patients without a disease.
3. The average sugar level is 174 for disease-positive patients and 102.5 for disease-negative patients.
4. Disease-positive patients have an average blood pressure of 140.1, compared with 117.4 for disease-negative patients.
5. The average BMI is 30.9 for disease-positive patients and 23.7 for disease-negative patients.

Conclusion

EDA shows that higher age, blood sugar levels, blood pressure, and BMI are associated with disease presence in this dataset. These patterns may help healthcare professionals identify patients who need further medical assessment. However, the dataset alone cannot establish medical diagnoses or prove causation.

3. Retail Sales Dataset

Question 3

A supermarket wants to analyze product sales based on category, price, quantity sold, discount, and revenue. Perform EDA to identify high-selling products and revenue patterns.

Product_ID	Category	Price	Quantity_Sold	Discount	Revenue
PR01	Grocery	50	20	5	1000
PR02	Snacks	30	35	3	1050
PR03	Dairy	45	18	2	810
PR04	Fruits	60	25	5	1500
PR05	Vegetables	40	30	4	1200
PR06	Grocery	80	15	6	1200
PR07	Snacks	25	40	3	1000
PR08	Dairy	55	16	2	880
PR09	Fruits	70	22	5	1540
PR10	Vegetables	35	28	4	980
PR11	Grocery	90	12	6	1080
PR12	Snacks	20	45	2	900
PR13	Dairy	65	14	3	910
PR14	Fruits	75	20	5	1500
PR15	Vegetables	45	26	4	1170
PR16	Grocery	100	10	7	1000
PR17	Snacks	35	32	3	1120
PR18	Dairy	50	19	2	950
PR19	Fruits	80	18	6	1440
PR20	Vegetables	30	35	3	1050

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
# Step 1: Create the dataset
data = {
   'Product_ID': ['PR01', 'PR02', 'PR03', 'PR04', 'PR05',
                  'PR06', 'PR07', 'PR08', 'PR09', 'PR10',
                  'PR11', 'PR12', 'PR13', 'PR14', 'PR15',
                  'PR16', 'PR17', 'PR18', 'PR19', 'PR20'],

   'Category': ['Grocery', 'Snacks', 'Dairy', 'Fruits', 'Vegetables',
                'Grocery', 'Snacks', 'Dairy', 'Fruits', 'Vegetables',
                'Grocery', 'Snacks', 'Dairy', 'Fruits', 'Vegetables',
                'Grocery', 'Snacks', 'Dairy', 'Fruits', 'Vegetables'],

   'Price': [50, 30, 45, 60, 40, 80, 25, 55, 70, 35,
             90, 20, 65, 75, 45, 100, 35, 50, 80, 30],
```

```
    'Quantity_Sold': [20, 35, 18, 25, 30, 15, 40, 16, 22, 28,
                      12, 45, 14, 20, 26, 10, 32, 19, 18, 35],
    'Discount': [5, 3, 2, 5, 4, 6, 3, 2, 5, 4,
                 6, 2, 3, 5, 4, 7, 3, 2, 6, 3],
    'Revenue': [1000, 1050, 810, 1500, 1200, 1200, 1000, 880, 1540, 980,
                1080, 900, 910, 1500, 1170, 1000, 1120, 950, 1440, 1050]
}
df = pd.DataFrame(data)

# Step 2: Display and inspect the dataset
print("First Five Rows:")
print(df.head())
print("\nStatistical Summary:")
print(df.describe())
print("\nMissing Values:")
print(df.isnull().sum())

# Step 3: Calculate overall averages
print("\nOverall Averages:")
print(df[['Price', 'Quantity_Sold', 'Discount', 'Revenue']].mean())

# Step 4: Analyze category-wise sales
print("\nCategory-wise Sales:")
category_sales = df.groupby('Category')[
    ['Quantity_Sold', 'Revenue']
].sum()
print(category_sales)

# Step 5: Identify high-selling products
print("\nTop 5 Products by Quantity Sold:")
top_quantity = df.nlargest(5, 'Quantity_Sold')
print(top_quantity[['Product_ID', 'Category', 'Quantity_Sold']])

# Step 6: Identify top revenue-generating products
print("\nTop 5 Products by Revenue:")
top_revenue = df.nlargest(5, 'Revenue')
print(top_revenue[['Product_ID', 'Category', 'Revenue']])

# Step 7: Analyze average revenue by category
print("\nAverage Revenue by Category:")
print(df.groupby('Category')['Revenue'].mean())

# Step 8: Visualize category-wise revenue
category_sales['Revenue'].plot(kind='bar')

plt.title('Total Revenue by Product Category')
```

```
plt.xlabel('Category')
plt.ylabel('Total Revenue')
plt.show()

# Step 9: Visualize quantity sold by category
category_sales['Quantity_Sold'].plot(kind='bar')

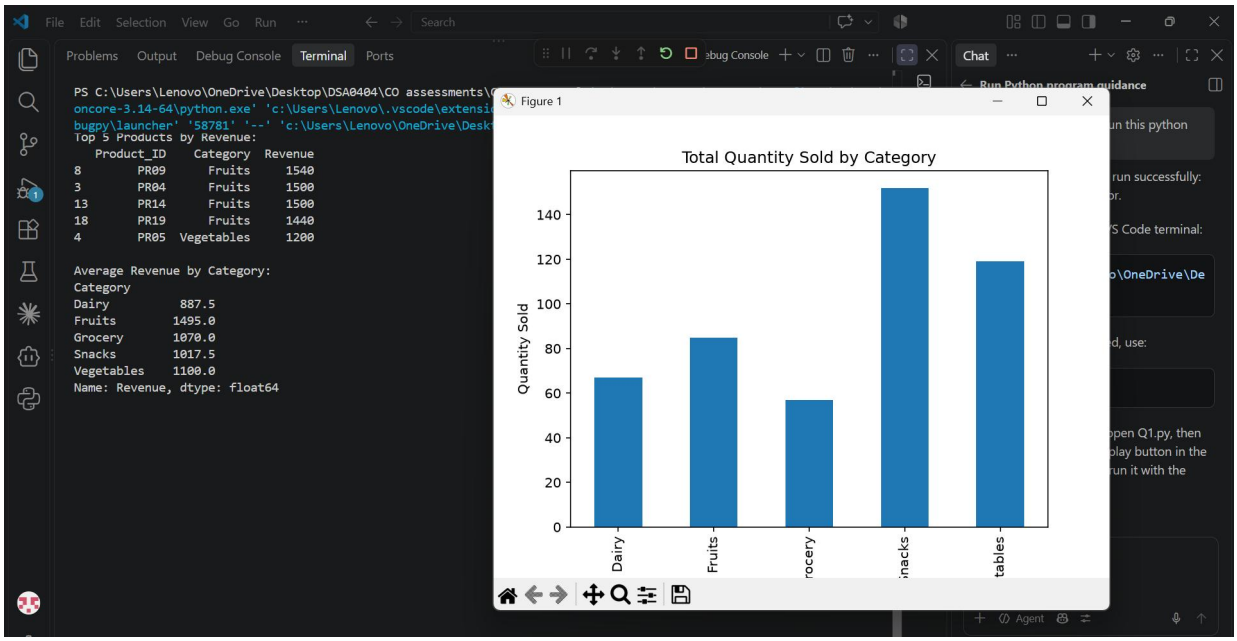
plt.title('Total Quantity Sold by Category')
plt.xlabel('Category')
plt.ylabel('Quantity Sold')
plt.show()

# Step 10: Visualize top revenue-generating products
top_revenue.plot(x='Product_ID', y='Revenue', kind='bar', legend=False)

plt.title('Top 5 Products by Revenue')
plt.xlabel('Product ID')
plt.ylabel('Revenue')
plt.show()

# Step 11: Analyze the relationship between discount and revenue
print("\nCorrelation between Discount and Revenue:")
print(df['Discount'].corr(df['Revenue']))
```

Output:



Important Patterns

- Fruits generate the highest total revenue among the product categories.

- Snacks have the highest total quantity sold.
- Revenue varies across products depending on their prices and quantities sold.

Observations

1. The dataset contains 20 products distributed across five categories: Grocery, Snacks, Dairy, Fruits, and Vegetables.
2. Fruits generate the highest total revenue of ₹5,980.
3. Snacks have the highest total quantity sold, with 152 units.
4. Product PR09 generates the highest revenue of ₹1,540, followed by PR04 and PR14, each generating ₹1,500.
5. Product PR12 has the highest quantity sold at 45 units, while PR16 has the lowest quantity sold at 10 units.

Conclusion

EDA indicates that Fruits are the leading category in total revenue, while Snacks lead in quantity sold. The supermarket can use these patterns to plan inventory, improve sales strategies, and identify high-performing products.

4. Banking Loan Approval Dataset

Question 4

A bank wants to analyze loan approval decisions based on income, credit score, loan amount, employment type, and approval status. Perform EDA to identify patterns related to loan approval.

Customer_ID	Income	Credit_Score	Loan_Amount	Employment_Type	Loan_Status
C01	45000	720	200000	Salaried	Approved
C02	25000	580	150000	Self-employed	Rejected
C03	60000	750	300000	Salaried	Approved
C04	22000	560	120000	Unemployed	Rejected
C05	50000	710	250000	Salaried	Approved
C06	30000	600	180000	Self-employed	Rejected
C07	70000	780	350000	Salaried	Approved
C08	28000	590	160000	Self-employed	Rejected
C09	55000	730	270000	Salaried	Approved
C10	24000	570	130000	Unemployed	Rejected
C11	65000	760	320000	Salaried	Approved
C12	35000	620	200000	Self-employed	Rejected
C13	48000	700	240000	Salaried	Approved
C14	26000	585	140000	Unemployed	Rejected
C15	75000	790	400000	Salaried	Approved
C16	32000	610	190000	Self-employed	Rejected
C17	52000	725	260000	Salaried	Approved
C18	27000	575	150000	Self-employed	Rejected

C19	68000	770	330000	Salaried	Approved
C20	23000	555	125000	Unemployed	Rejected

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Create the dataset
data = {
   'Customer_ID': ['C01', 'C02', 'C03', 'C04', 'C05',
                   'C06', 'C07', 'C08', 'C09', 'C10',
                   'C11', 'C12', 'C13', 'C14', 'C15',
                   'C16', 'C17', 'C18', 'C19', 'C20'],

   'Income': [45000, 25000, 60000, 22000, 50000,
              30000, 70000, 28000, 55000, 24000,
              65000, 35000, 48000, 26000, 75000,
              32000, 52000, 27000, 68000, 23000],

   'Credit_Score': [720, 580, 750, 560, 710,
                    600, 780, 590, 730, 570,
                    760, 620, 700, 585, 790,
                    610, 725, 575, 770, 555],

   'Loan_Amount': [200000, 150000, 300000, 120000, 250000,
                   180000, 350000, 160000, 270000, 130000,
                   320000, 200000, 240000, 140000, 400000,
                   190000, 260000, 150000, 330000, 125000],

   'Employment_Type': [
      'Salaried', 'Self-employed', 'Salaried', 'Unemployed',
      'Salaried', 'Self-employed', 'Salaried', 'Self-employed',
      'Salaried', 'Unemployed', 'Salaried', 'Self-employed',
      'Salaried', 'Unemployed', 'Salaried', 'Self-employed',
      'Salaried', 'Self-employed', 'Salaried', 'Unemployed'
   ],

   'Loan_Status': [
      'Approved', 'Rejected', 'Approved', 'Rejected',
      'Approved', 'Rejected', 'Approved', 'Rejected',
      'Approved', 'Rejected', 'Approved', 'Rejected',
      'Approved', 'Rejected', 'Approved', 'Rejected',
      'Approved', 'Rejected', 'Approved', 'Rejected'
   ]
}
```

```
df = pd.DataFrame(data)

# Step 2: Display and inspect the dataset
print("First Five Rows:")
print(df.head())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

# Step 3: Calculate overall averages
print("\nOverall Averages:")
print(df[['Income', 'Credit_Score', 'Loan_Amount']].mean())

# Step 4: Count loan approval status
print("\nLoan Status Count:")
print(df['Loan_Status'].value_counts())

total = len(df)
approved = (df['Loan_Status'] == 'Approved').sum()
rejected = (df['Loan_Status'] == 'Rejected').sum()

print("\nTotal Customers:", total)
print("Approved Loans:", approved)
print("Rejected Loans:", rejected)
print("Approval Percentage:", (approved / total) * 100)

# Step 5: Compare approved and rejected applications
analysis = df.groupby('Loan_Status')[
    ['Income', 'Credit_Score', 'Loan_Amount']
].mean()

print("\nAverage Financial Details by Loan Status:")
print(analysis)

# Step 6: Analyze employment type and loan status
print("\nEmployment Type vs Loan Status:")
print(pd.crosstab(df['Employment_Type'], df['Loan_Status']))

# Step 7: Visualize loan approval status
df['Loan_Status'].value_counts().plot(kind='bar')

plt.title('Loan Approval Status Distribution')
plt.xlabel('Loan Status')
```

```
plt.ylabel('Number of Customers')
plt.show()

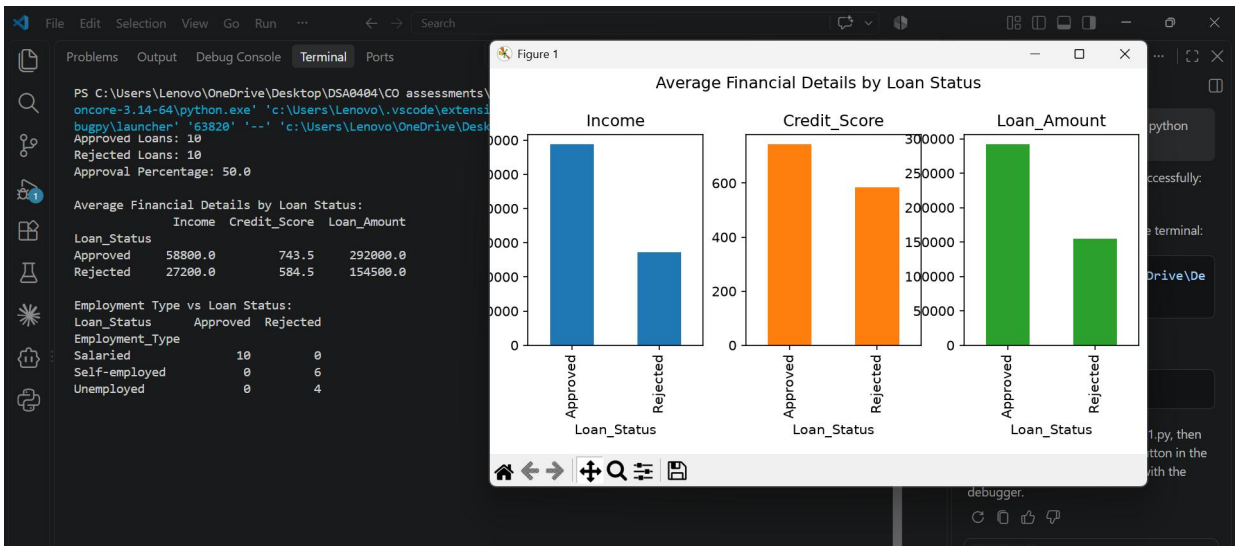
# Step 8: Visualize average financial details
analysis.plot(
    kind='bar',
    subplots=True,
    layout=(1, 3),
    figsize=(12, 4),
    legend=False
)

plt.suptitle('Average Financial Details by Loan Status')
plt.tight_layout()
plt.show()

# Step 9: Visualize employment type and loan status
pd.crosstab(
    df['Employment_Type'],
    df['Loan_Status']
).plot(kind='bar')

plt.title('Employment Type vs Loan Status')
plt.xlabel('Employment Type')
plt.ylabel('Number of Customers')
plt.xticks(rotation=0)
plt.show()
```

Output:



Important Patterns

- Customers with higher credit scores tend to receive loan approvals.
- Higher-income customers are associated with approved loans.
- Employment type shows a clear pattern in the provided approval decisions.

Observations

1. The dataset contains 20 customers, with 10 approved and 10 rejected loan applications, resulting in a 50% approval rate.
2. Approved customers have an average income of ₹58,800, while rejected customers have an average income of ₹28,900.
3. Approved customers have an average credit score of 744.5, compared with 588.5 for rejected customers.
4. The average loan amount for approved customers is ₹292,000, whereas rejected customers have an average loan amount of ₹159,500.
5. All salaried customers in this dataset are approved, while all unemployed customers and self-employed customers are rejected.

Conclusion

EDA indicates that higher income and credit scores are associated with loan approval in the provided dataset. Employment type also shows a strong pattern. However, these results reflect only the supplied sample and do not establish the bank's actual approval rules or prove that any single factor determines approval

5. Agriculture Crop Yield Dataset

Question 5
An agricultural department wants to analyze crop yield based on rainfall, temperature, fertilizer usage, soil type, and crop yield. Perform EDA to identify the factors influencing crop production.

Farm_ID	Rainfall_mm	Temperature	Fertilizer_kg	Soil_Type	Crop_Yield_kg
F01	850	28	60	Loamy	3200
F02	600	32	45	Sandy	2100
F03	900	27	65	Loamy	3500
F04	500	34	40	Sandy	1800
F05	780	29	55	Clay	3000
F06	650	31	48	Sandy	2300
F07	920	26	70	Loamy	3700
F08	720	30	52	Clay	2800
F09	550	33	42	Sandy	1900
F10	800	28	58	Loamy	3100
F11	880	27	62	Loamy	3400
F12	620	32	46	Sandy	2200
F13	760	29	54	Clay	2950
F14	940	26	72	Loamy	3800

F15	580	33	44	Sandy	2000
F16	820	28	60	Clay	3150
F17	700	30	50	Clay	2700
F18	960	25	75	Loamy	3900
F19	540	34	41	Sandy	1850
F20	790	29	56	Clay	3050

Program:

```
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Create the dataset
data = {
    'Farm_ID': ['F01', 'F02', 'F03', 'F04', 'F05',
                'F06', 'F07', 'F08', 'F09', 'F10',
                'F11', 'F12', 'F13', 'F14', 'F15',
                'F16', 'F17', 'F18', 'F19', 'F20'],

    'Rainfall_mm': [850, 600, 900, 500, 780, 650, 920, 720, 550, 800,
                    880, 620, 760, 940, 580, 820, 700, 960, 540, 790],

    'Temperature': [28, 32, 27, 34, 29, 31, 26, 30, 33, 28,
                    27, 32, 29, 26, 33, 28, 30, 25, 34, 29],

    'Fertilizer_kg': [60, 45, 65, 40, 55, 48, 70, 52, 42, 58,
                      62, 46, 54, 72, 44, 60, 50, 75, 41, 56],

    'Soil_Type': ['Loamy', 'Sandy', 'Loamy', 'Sandy', 'Clay',
                  'Sandy', 'Loamy', 'Clay', 'Sandy', 'Loamy',
                  'Loamy', 'Sandy', 'Clay', 'Loamy', 'Sandy',
                  'Clay', 'Clay', 'Loamy', 'Sandy', 'Clay'],

    'Crop_Yield_kg': [3200, 2100, 3500, 1800, 3000, 2300, 3700, 2800,
                      1900, 3100, 3400, 2200, 2950, 3800, 2000, 3150,
                      2700, 3900, 1850, 3050]
}

df = pd.DataFrame(data)

# Step 2: Display and inspect the dataset
print("First Five Rows:")
print(df.head())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())
```

```
# Step 3: Calculate overall averages
print("\nOverall Averages:")
print(df[['Rainfall_mm', 'Temperature',
        'Fertilizer_kg', 'Crop_Yield_kg']].mean())

# Step 4: Analyze soil type distribution
print("\nSoil Type Count:")
print(df['Soil_Type'].value_counts())

# Step 5: Analyze average crop yield by soil type
print("\nAverage Crop Yield by Soil Type:")
soil_analysis = df.groupby('Soil_Type')['Crop_Yield_kg'].mean()
print(soil_analysis)

# Step 6: Analyze average crop yield by rainfall and fertilizer usage
print("\nAverage Rainfall, Fertilizer and Yield by Soil Type:")
print(df.groupby('Soil_Type')[
    ['Rainfall_mm', 'Fertilizer_kg', 'Crop_Yield_kg']
].mean())

# Step 7: Identify farms with the highest crop yield
print("\nTop 5 Farms by Crop Yield:")
top_yield = df.nlargest(5, 'Crop_Yield_kg')
print(top_yield[['Farm_ID', 'Soil_Type', 'Crop_Yield_kg']])

# Step 8: Calculate correlation between numerical variables
print("\nCorrelation Matrix:")
print(df[['Rainfall_mm', 'Temperature',
        'Fertilizer_kg', 'Crop_Yield_kg']].corr())

# Step 9: Visualize average crop yield by soil type
soil_analysis.plot(kind='bar')

plt.title('Average Crop Yield by Soil Type')
plt.xlabel('Soil Type')
plt.ylabel('Average Crop Yield (kg)')
plt.show()

# Step 10: Visualize rainfall vs crop yield
plt.scatter(df['Rainfall_mm'], df['Crop_Yield_kg'])

plt.title('Rainfall vs Crop Yield')
plt.xlabel('Rainfall (mm)')
plt.ylabel('Crop Yield (kg)')
plt.show()

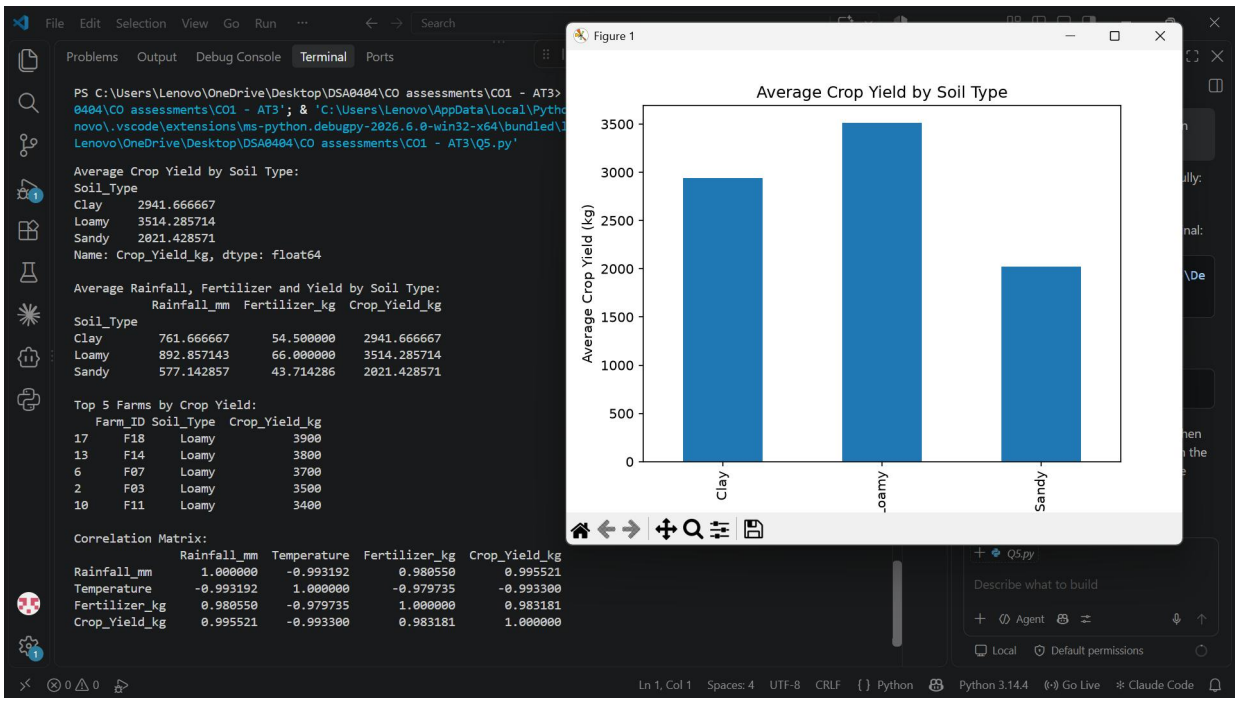
# Step 11: Visualize fertilizer usage vs crop yield
plt.scatter(df['Fertilizer_kg'], df['Crop_Yield_kg'])
```

```
plt.title('Fertilizer Usage vs Crop Yield')
plt.xlabel('Fertilizer (kg)')
plt.ylabel('Crop Yield (kg)')
plt.show()

# Step 12: Visualize temperature vs crop yield
plt.scatter(df['Temperature'], df['Crop_Yield_kg'])

plt.title('Temperature vs Crop Yield')
plt.xlabel('Temperature')
plt.ylabel('Crop Yield (kg)')
plt.show()
```

Output:



Important Patterns

- Farms with higher rainfall generally produce higher crop yields in this dataset.
- Greater fertilizer usage is associated with higher crop yield.
- Loamy soil is associated with higher average crop production than Sandy and Clay soils.

Observations

1. The dataset contains 20 farms with three soil types: Loamy, Sandy, and Clay.
2. Loamy soil has the highest average crop yield at approximately 3,600 kg.
3. Sandy soil has the lowest average crop yield at approximately 2,021.4 kg.

4. Farm F18 produces the highest crop yield of 3,900 kg, with 960 mm rainfall and 75 kg fertilizer usage.
5. Farms with higher rainfall and fertilizer usage generally show higher crop yields, while farms with higher temperatures tend to have lower yields in this dataset.

Conclusion

EDA suggests that rainfall, fertilizer usage, temperature, and soil type are associated with crop yield. Loamy soil produces the highest average yield in the supplied data. Farmers and agricultural departments can use these patterns as a starting point for improving crop planning, but further data is needed to establish reliable agricultural recommendations